Stats 506 F25 - Answer Key

Worked reference answers. Match the year and problem label before checking your work. Minor errata are clarified where relevant.

https://dept.stat.lsa.umich.edu/~jerrick/courses/stat506_f25/midtermsolns.html

Problem 1

a.

q = 25: the retained values are 1, 3, 5, 7, 9.

b.

q = 24: x %% 3 == 0 selects the third column, containing 7, 8, 9.

c.

```
     [,1] [,2]
[1,]    2    2
[2,]   12   12
[3,]   30   30
[4,]   56   56
```

Recycling proceeds down each matrix column: multiply 1,3,5,7 by 2,4,6,8.

d.

```
c(1, 2, 3, 4, 0, 1, 2, 3, 4, 5)
```

e.

q = 2. The grouped v values are 1 for y=2 and 2 for y=3; ascending order keeps y=2.

f.

q = 15. bar(5) calls baz(30), which returns 15. foo is not called.

Problem 2

One valid approach follows. Here the analysis requires known sex and age in e-f; NA values in numeric transformations remain NA.

a.

```
num <- vapply(mydata, is.numeric, logical(1))
mydata[num] <- lapply(mydata[num], function(x) {
  x[x < 0] <- NA
  x
})
```

Operate only on numeric columns, not character columns.

b.

```
mydata$mean_arterial_pressure <-
  (2 * mydata$diastolic_bp + mydata$systolic_bp) / 3
```

c.

```
mydata$smoking <- mydata$years_smoking > 0
```

d.

```
mydata$female <- mydata$sex == "Female"
mydata$female[mydata$sex == ""] <- NA
```

e.

```
mydata <- mydata[which(mydata$female), , drop = FALSE]
```

f.

```
keep <- !is.na(mydata$age) & mydata$age >= 18 & mydata$age <= 65
mydata <- mydata[keep, , drop = FALSE]
```

g.

```
lo <- min(mydata$outcome_score, na.rm = TRUE)
hi <- max(mydata$outcome_score, na.rm = TRUE)
mydata$outcome_score <- (mydata$outcome_score - lo) / (hi - lo)
```

Assume at least two distinct nonmissing values. The official unguarded min/max expressions need na.rm=TRUE when missing outcomes remain.

h.

```
mod <- lm(outcome_score ~ factor(treatment) +
            age * mean_arterial_pressure +
            cholesterol + I(cholesterol^2) + I(cholesterol^3),
          data = mydata)
```

i.

```
mean(predict(mod))
```

This averages predictions on the observations used by the fitted model.

Problem 3

a.

```
noisyx <- x + rnorm(length(x))
```

rnorm(1) supplies a single shared random shift; it is not the constant 1.

b.

```
data$yearsempl <- data$yearsfull
data$yearsempl[data$isparttime] <-
  data$yearspart[data$isparttime]
```

Subset both sides by the same part-time indicator.

c.

```
total <- 0
for (i in seq_along(data)) {
  total <- total + data[i]
}
```

Initialize the accumulator outside the loop. A different name also avoids hiding sum().

d.

```
cars %>%
  group_by(manufacturer) %>%
  summarize(meanmpg = mean(mpg), .groups = "drop")
```

The original code computes one overall mean because group_by is missing.

e.

Always TRUE: each ordinary data.frame column has the same number of rows.

f.

Always FALSE: replacing values above 50 by 50 can only decrease or preserve the mean, not increase it.

g.

Depends. Equality is possible when every original row survives the rural filter and every state has one row; otherwise aggregation or filtering may reduce the row count.

Problem 4

a

p	q	r	k
3	5	1	2
9	5	2	4

b

k	p2
4	11
1	8

c

r	pp
1	9
2	9

d

p	m
6	10
3	20
9	40
2	60

Problem 4 (continued)

e

p	m
6	10
3	20
9	40
2	60
NA	30
NA	50

f.

```
SELECT o.r, o.k, s.m AS sku_value
FROM orders AS o
LEFT JOIN sku_map AS s ON o.k = s.k
ORDER BY o.k DESC
```

Problem 5

X means the pattern matches somewhere in the string, subject to any anchors. Each pattern is displayed as R string contents.

```
a. "^[^@]+@.+\\..+"
b. "(org|com)"
c. "[0-9]{3}"
d. "\\.[a-z]+\\.[a-z]+$"
```

String	a	b	c	d
alice@gmail.com	X	X		
bob.smith@umich.edu	X			
test123@yahoo.com	X	X	X	
jane-doe@company.org	X	X		
contact@123web.net	X		X	
admin@mailserver.local	X			
user99@gmail.com	X	X		
info@college.edu	X			
myorg@domain.co.uk	X	X		X
support@website.io	X			

a matches all ten. b looks for org or com anywhere, so myorg@domain.co.uk qualifies through the local part myorg. c requires three consecutive digits, not merely any three digits. d matches two dotted alphabetic components at the end.

Problem 6

a.

```
open_pack <- function(pack_type) {
  probs <- if (pack_type == 1) rep(0.2, 5) else
    c(0.3, 0.3, 0.3, 0.05, 0.05)
  sample(LETTERS[1:5], 1, replace = TRUE, prob = probs)
}
```

b.

```
open_packs <- function(pack_type, n) {
  probs <- if (pack_type == 1) rep(0.2, 5) else
    c(0.3, 0.3, 0.3, 0.05, 0.05)
  sample(LETTERS[1:5], n, replace = TRUE, prob = probs)
}
```

c.

```
spend40 <- function(pack_type, reps = 10000) {
  n <- if (pack_type == 1) 8 else 20
  draws <- matrix(open_packs(pack_type, n * reps), nrow = n)
  complete <- apply(draws, 2, function(x) length(unique(x)) == 5)
  100 * mean(complete)
}
set.seed(506)
spend40(1)
spend40(2)
```

Pack 2 performs better: estimates are roughly 32% versus 40%. Every column is one budget-sized experiment. The official code returns a fraction via mean(complete); multiplying by 100 makes the returned value a percentage as requested.